

Ampliación de Programación en Python

DAW V



UT01 – Elementos de un programa informático

Profesora: Julia Gallego Sánchez (julia.gs@educa.madrid.org)
IES Virgen de la Paz – Alcobendas

El lenguaje Python

- Fácil de aprender
- Rápido para generar código nuevo
- Fácil de obtener, instalar e implementar
- Gratuito, abierto y multiplataforma





El lenguaje Python

Python es un **lenguaje de programación de alto nivel, interpretado y multiparadigma**, diseñado para ser **fácil de leer, escribir y entender**.

A diferencia de los lenguajes compilados tradicionales (como C o Java), Python **no se ejecuta directamente como código máquina**, sino que pasa por un **proceso intermedio**:

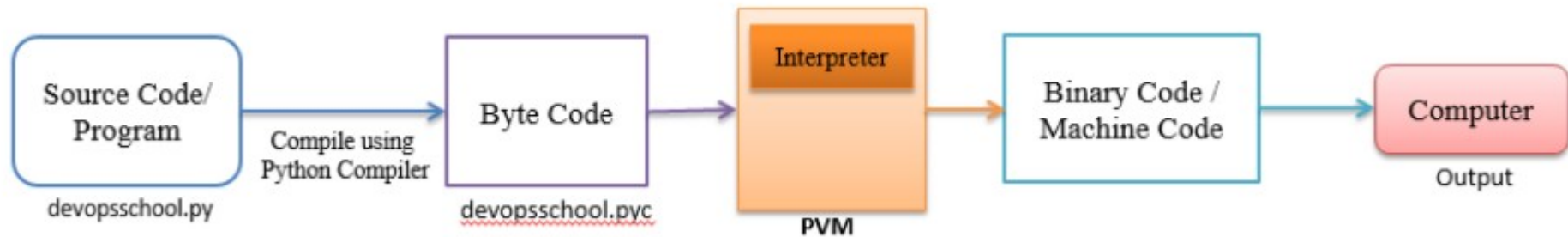
- 1** El **código fuente** (.py) se traduce a un **bytecode**, una forma más sencilla y compacta de las instrucciones.
- 2** Ese bytecode es interpretado por el **intérprete de Python** (la *máquina virtual de Python*, también llamada **PVM – Python Virtual Machine**).

De esta forma, Python combina la **portabilidad** de los lenguajes interpretados con cierta **eficiencia** gracias al bytecode intermedio.

Por eso, un mismo programa puede ejecutarse en distintos sistemas operativos sin modificar su código.

Python es multiparadigma porque permite elegir cómo resolver un problema: puedes escribir código estructurado, orientado a objetos o funcional, e incluso combinarlos en el mismo programa.

El lenguaje Python



El lenguaje Python

🧩 Qué necesitamos para ejecutar un programa en Python

◆ 1 El intérprete de Python

Python no se ejecuta directamente como el código máquina, por eso necesitamos tener instalado el **intérprete de Python**, que es el programa encargado de **leer, traducir y ejecutar el código**.

El intérprete incluye:

- El **compilador**, que convierte el código fuente (`.py`) en **bytecode** (`.pyc`).
- La **PVM (Python Virtual Machine)**, que interpreta ese bytecode y ejecuta las instrucciones.

👉 Puedes descargarlo desde la página oficial: <https://www.python.org> ↗

El lenguaje Python

◆ 2 Un editor o entorno de desarrollo (IDE)

Necesitamos un programa donde escribir el código.

Algunas opciones son:

Tipo	Ejemplos
Editores simples	Bloc de notas, Notepad++, Sublime Text
Entornos de desarrollo (IDE)	Visual Studio Code, PyCharm, Thonny, IDLE (viene con Python)

Para ejecutar un programa en Python necesitamos:

- “Tener instalado el intérprete de Python.”
- “Usar un editor o entorno de desarrollo donde escribir el código.”
- “Ejecutar el archivo `.py` con el intérprete, que lo convierte en bytecode y lo ejecuta mediante la máquina virtual (PVM).”



El lenguaje Python

Ahora que sabemos cómo ejecutar Python, vamos a descubrir cómo se escribe correctamente el código.

Convenciones de nombres en Python

🌱 Convenciones de nombres en Python (según PEP 8)

Elemento	Estilo en Python	Ejemplos correctos ✅	Ejemplos incorrectos ❌	Comentarios
Nombre del archivo (.py)	minúsculas_con_guiones_bajos	<code>ejercicio1.py</code> <code>mi_programa_print.py</code>	<code>Ejercicio1.py</code> <code>MiPrograma.py</code>	Evita mayúsculas y espacios. Usa <code>_</code> para separar palabras.
Variable	minúsculas_con_guiones_bajos	<code>nombre</code> , <code>edad</code> , <code>color_favorito</code>	<code>Nombre</code> , <code>EdadAlumno</code>	Descriptivas y cortas; todas en minúsculas.
Función	minúsculas_con_guiones_bajos	<code>calcular_media()</code> , <code>mostrar_resultado()</code>	<code>CalcularMedia()</code> , <code>MostrarResultado()</code>	Igual que las variables; los paréntesis indican que es función.
Clase	MayúsculaInicial (CamelCase)	<code>Alumno</code> , <code>CuentaBancaria</code> , <code>VehiculoDeportivo</code>	<code>alumno</code> , <code>cuenta_bancaria</code>	Cada palabra empieza con mayúscula (como en Java).
Constante	MAYÚSCULAS_CON_GUIONES_BAJOS	<code>PI = 3.1416</code> , <code>MAX_ALUMNOS = 30</code>	<code>pi</code> , <code>MaxAlumnos</code>	Se usan para valores fijos o que no cambian.
Módulo / Paquete	minúsculas_con_guiones_bajos	<code>utilidades</code> , <code>mi_paquete</code>	<code>MiPaquete</code> , <code>PaqueteUtilidades</code>	Mismo formato que los archivos <code>.py</code> .
Comentario / Docstring	Frase en minúscula, clara y breve	<code># calcula el promedio de las notas</code>	<code># CALCULA EL PROMEDIO</code>	En minúsculas, concisos y explicativos.

Funciones y sobrecarga

- Una **función** (por ejemplo, “print”) es un fragmento de código que:
 - Recibe unos argumentos
 - Produce un efecto
 - Devuelve un resultado
- “**Sobrecarga**” es la posibilidad de que una función realice diferentes acciones según el número de argumentos que recibe

```
>>> print("Hello", "World")  
Hello World
```

```
>>> print("Hello", "World", end="*")  
Hello World*
```

```
>>> print("Hello", "World", sep="-")  
Hello-World
```

```
>>> c='Esto es\nun string'  
>>> print(c)  
Esto es un  
string
```

Ejemplos de la función print() en Python

Nº	Descripción	Ejemplo
1	Texto simple	<code>print("Hola mundo")</code>
2	Varios argumentos	<code>print("Hola", "soy", "Python")</code>
3	Texto + número	<code>print("Tengo", 3, "manzanas")</code>
4	Separador con guion	<code>print("Año", 2025, sep="-")</code>
5	Separador con coma	<code>print("Lunes", "Martes", sep=", ")</code>
6	Separador con salto	<code>print("Rojo", "Verde", sep="\n")</code>
7	Sin salto de línea	<code>print("Hola", end=""); print("mundo")</code>
8	Final personalizado	<code>print("Hola", end="!!!"); print("Mundo")</code>
9	sep y end combinados	<code>print("Python", "Java", sep=" ", end=" --> Fin")</code>
10	Con variables	<code>print("Mi nombre es", nombre, "y tengo", edad, "años.")</code>
11	f-string simple	<code>print(f"Hola, me llamo {nombre}")</code>
12	f-string con operación	<code>print(f"La suma es {a+b}")</code>
13	Varios saltos de línea	<code>print("Lunes\nMartes\nMiércoles")</code>
14	Comillas dobles dentro	<code>print('Ella dijo: "Hola"')</code>
15	Comillas simples dentro	<code>print("Él respondió: 'Buenos días'")</code>
16	Tabulación	<code>print("Nombre\tEdad\tPaís")</code>

Nota: Caracteres especiales (escape “\” cambia el significado del carácter que viene justo después)

`\t` – Tabulador

`\n` – Retorno de carro

Ejercicios prácticos sobre la función print() y la sobrecarga de argumentos

Actividad práctica

Programa en **Visual Studio Code** utilizando el lenguaje **Python** todas las opciones mostradas en la tabla anterior. Recuerda usar las convenciones de nombres en Python para todo aquello que necesites.

Cada fila representa un ejemplo diferente del uso de la función print().

Tu tarea consiste en escribir el código correspondiente, ejecutarlo y comprobar el resultado en la consola.

Asegúrate de:

- Usar correctamente los argumentos sep y end cuando se indiquen.
- Combinar texto, números y variables según el ejemplo.
- Probar las versiones con comillas, saltos de línea (\n) y tabulaciones (\t).

Guardar el archivo con el nombre: ejercicios_print.py



Tipos numéricos en Python

Tipos numéricos en Python

Tipo	Descripción	Ejemplo de código	Resultado / Valor	Notas importantes
int	Número entero , positivo o negativo, sin decimales.	<pre>x = 10 y = -3</pre>	10 , -3	Es el tipo más común para contar o indexar.
long	(📄 Solo existía en Python 2) Número entero grande , sin límite fijo.	<pre>x = 1234567890123456789L</pre>	1234567890123456789	❌ En Python 3 , todos los enteros son automáticamente <code>int</code> , sin límite de tamaño.
float	Número decimal , con parte fraccionaria.	<pre>x = 3.14 y = -0.5</pre>	3.14 , -0.5	Se usa para cálculos con decimales. Cuidado con los redondeos.
complex	Número complejo , con parte real e imaginaria.	<pre>x = 2 + 3j y = 1 - 4j</pre>	(2+3j) , (1-4j)	<code>j</code> representa la unidad imaginaria $\sqrt{-1}$.
Conversión entre tipos	Puedes transformar un número de un tipo a otro.	<pre>float(5) → 5.0 int(3.9) → 3</pre>		Usa funciones como <code>int()</code> , <code>float()</code> , <code>complex()</code>

Operadores Aritméticos en Python



1

Operadores Aritméticos en Python

Operador	Ejemplo	Resultado	Descripción	
+	<code>5 + 3</code>	<code>8</code>	Suma dos valores.	
-	<code>10 - 4</code>	<code>6</code>	Resta el segundo valor al primero.	
*	<code>2 * 5</code>	<code>10</code>	Multiplica dos valores.	
/	<code>8 / 2</code>	<code>4.0</code>	División con resultado decimal (<code>float</code>).	
//	<code>8 // 3</code>	<code>2</code>	División entera (descarta decimales).	
%	<code>10 % 3</code>	<code>1</code>	Devuelve el resto de la división.	
**	<code>2 ** 3</code>	<code>8</code>	Potencia: eleva el primer número al segundo.	



Recuerda:

Si la división usa `/`, el resultado siempre es **decimal**, aunque sea exacto.

Operaciones con Strings

Tipo de dato `str` – Cadenas de texto en Python

Tipo de dato	Descripción	Ejemplo	Resultado / Explicación
<code>str</code>	Representa texto formado por letras, números o símbolos. Se escribe entre comillas simples <code>' '</code> o dobles <code>" "</code> .	<pre>nombre = "Ana" print("Hola", nombre)</pre>	<code>Hola Ana</code> La variable contiene una cadena de texto .

Operación	Descripción	Ejemplo de código	Resultado / Explicación
Concatenación (+)	Une dos cadenas	<code>"Hola " + "Mundo"</code>	Hola Mundo
Repetición (*)	Repite una cadena varias veces	<code>"ha" * 3</code>	hahaha
Longitud (len())	Cuenta los caracteres	<code>len("Python")</code>	6
Indexación []	Accede a un carácter	<code>"Python"[0]</code>	P
Slicing [:]	Extrae una parte	<code>"Python"[0:3]</code>	Pyt
Mayúsculas .upper()	Convierte a mayúsculas	<code>"hola".upper()</code>	HOLA
Minúsculas .lower()	Convierte a minúsculas	<code>"HOLA".lower()</code>	hola
Capitalizar .capitalize()	Primera letra mayúscula	<code>"python".capitalize()</code>	Python
Título .title()	Primera letra de cada palabra	<code>"hola mundo".title()</code>	Hola Mundo
Eliminar espacios .strip()	Elimina espacios	<code>" hola ".strip()</code>	hola
Buscar .find()	Posición del texto	<code>"programar".find("gra")</code>	3
Reemplazar .replace()	Sustituye texto	<code>"hola mundo".replace("mundo", "Python")</code>	hola Python
Comprobar (in)	Verifica si existe un texto	<code>"Py" in "Python"</code>	True
Dividir .split()	Separa palabras en lista	<code>"uno dos tres".split()</code>	<code>['uno', 'dos', 'tres']</code>
Unir .join()	Une lista con separador	<code>" - ".join(["uno", "dos", "tres"])</code>	uno - dos - tres
Formato f-string	Inserta variables	<code>nombre="Ana"; f"Hola {nombre}"</code>	Hola Ana



Ejercicios prácticos sobre operaciones con Strings

Actividad práctica: Strings en Python

Programa en Visual Studio Code utilizando el lenguaje Python todas las opciones mostradas en la siguiente tabla de ejercicios con strings. Guarda tu nuevo programa con el nombre `operaciones_strings.py`.

Cada fila representa un ejemplo diferente de uso de cadenas (str): concatenación, longitud, mayúsculas/minúsculas, slicing, búsqueda, reemplazo, división/unión, comparación, f-strings, etc.

En cada ejercicio, muestra exactamente lo indicado en la columna “Qué debe mostrar”, utilizando el comando/método sugerido.

Ejercicios prácticos sobre operaciones con Strings

🌱 Ejercicios con Strings en Python (1 al 8)

Nº	Descripción	Comando / Método sugerido	Qué debe mostrar
1	Concatenar texto	<code>+</code> , <code>print()</code>	Tu nombre y apellido juntos en una sola línea.
2	Longitud del texto	<code>len()</code>	El número total de caracteres de una frase introducida por el usuario.
3	Mayúsculas y minúsculas	<code>.upper()</code> , <code>.lower()</code> , <code>.capitalize()</code>	La frase en mayúsculas, minúsculas y con la primera letra en mayúscula.
4	Repetición de texto	<code>*</code> , <code>print()</code>	El nombre del usuario repetido 3 veces, separado por espacios.
5	Extraer caracteres (slicing)	<code>[]</code> , <code>[inicio:fin]</code>	La primera letra, la última y las tres primeras de <code>"PYTHON"</code> .
6	Buscar palabra en frase	<code>in</code> , <code>.find()</code>	Si la palabra está dentro de la frase (<code>True</code> o <code>False</code>).
7	Reemplazar texto	<code>.replace()</code>	Una frase con todas las apariciones de <code>"Python"</code> cambiadas por <code>"Java"</code> .
8	Comprobar inicio o final	<code>.startswith()</code> , <code>.endswith()</code>	Si el texto empieza por <code>"Hola"</code> o termina con <code>"Python"</code> .

Ejercicios prácticos sobre operaciones con Strings

✿ Ejercicios con Strings en Python (9 al 16)

Nº	Descripción	Comando / Método sugerido	Qué debe mostrar
9	Contar letras	<code>.count()</code>	Cuántas veces aparece una letra específica en la frase.
10	Invertir texto	<code>[::-1]</code>	La palabra al revés (por ejemplo, "Python" → "nohtyP").
11	Quitar espacios	<code>.strip()</code>	La frase sin espacios al principio ni al final.
12	Dividir y unir texto	<code>.split()</code> , <code>.join()</code>	La frase separada por palabras y luego unida con guiones (-).
13	Comparar palabras	<code>.lower()</code> , <code>==</code>	Si dos palabras son iguales sin importar mayúsculas/minúsculas.
14	f-string	<code>f"{}"</code>	Frase con nombre y edad: "Hola [nombre], el año que viene tendrás [edad+1]" .

Tipo de dato Booleano o bool

🧩 Tipo de dato: `bool`

◆ Definición

El tipo de dato **booleano** (`bool`) solo puede tener **dos posibles valores**:

Valor	Significado
<code>True</code>	Verdadero
<code>False</code>	Falso

Operadores de comparación

⚖️ Operadores de Comparación y Combinación Lógica en Python (números y cadenas)

Operador	Descripción	Ejemplo con números	Resultado	Ejemplo con strings	Resultado	📄
<code>==</code>	Igual a	<code>5 == 5</code>	True	<code>"hola" == "hola"</code>	True	
<code>!=</code>	Distinto de	<code>5 != 3</code>	True	<code>"hola" != "adiós"</code>	True	
<code>></code>	Mayor que	<code>7 > 3</code>	True	<code>"pera" > "manzana"</code>	True (<i>"p" va después de "m"</i>)	
<code><</code>	Menor que	<code>4 < 2</code>	False	<code>"gato" < "zorro"</code>	True (<i>"g" va antes de "z"</i>)	
<code>>=</code>	Mayor o igual que	<code>5 >= 5</code>	True	<code>"casa" >= "casa"</code>	True	
<code><=</code>	Menor o igual que	<code>6 <= 8</code>	True	<code>"azul" <= "rojo"</code>	True (<i>"a" va antes de "r"</i>)	
<code>and</code>	Verdadero si ambas condiciones son verdaderas	<code>(5 > 3) and (8 > 2)</code>	True	<code>("pera" > "manzana") and ("hola" == "hola")</code>	True	
<code>or</code>	Verdadero si una o más condiciones son verdaderas	<code>(5 > 10) or (8 > 2)</code>	True	<code>("pera" == "manzana") or ("hola" == "hola")</code>	True	

- Los operadores de comparación en Python siempre devuelven un valor booleano, es decir: solo pueden dar como resultado True o False.
- Estos operadores sí funcionan con texto (str), pero comparan por orden alfabético (lexicográfico).
- Mayúsculas y minúsculas son diferentes: `"Python" < "python" → True` (la "P" mayúscula vale menos que la "p").
- Si se desea comparar longitud, se usa `len()` → `len("hola") > len("adiós") → True`

Precedencia o prioridad de operadores

Precedencia de Operadores en Python (simplificada y completa)

Nivel	Tipo de operador	Ejemplo	Cómo se evalúa	
1	Paréntesis <code>()</code>	<code>(2 + 3) * 4</code>	Los paréntesis siempre se ejecutan primero . Sirven para cambiar el orden natural.	
2	Potencia <code>**</code>	<code>2 ** 3</code> → 8	Tiene más prioridad que cualquier otra operación aritmética. Se evalúa de derecha a izquierda .	
3	Multiplicación, división, módulo y división entera <code>*</code> , <code>/</code> , <code>%</code> , <code>//</code>	<code>10 / 2 * 3</code> → 15.0	Todos tienen la misma prioridad y se evalúan de izquierda a derecha .	
4	Suma y resta <code>+</code> , <code>-</code>	<code>10 - 2 + 4</code> → 12	También tienen la misma prioridad y se evalúan de izquierda a derecha .	

Las Variables en Python

◆ ¿Qué es una variable?

Una variable es un espacio en la memoria del ordenador donde se guarda un valor con un nombre identificativo.

Sirve para almacenar datos que luego se pueden usar, modificar o mostrar en el programa.

Ejemplo simple:

```
nombre = "Ana"  
edad = 20  
print(nombre, edad)
```

Salida

Ana 20

◆ Características principales

En Python no es necesario declarar el tipo de variable, el tipo se asigna automáticamente según el valor.

```
x = 5          # int  
y = "Hola"     # str  
z = 3.14       # float
```

Las variables pueden cambiar de valor durante la ejecución:

```
edad = 20  
edad = 21      # ahora vale 21
```

Las Variables en Python

◆ Reglas para nombrar variables:

- Deben empezar con una letra o guion bajo (_).
- Solo pueden contener letras, números y guiones bajos.
- Son sensibles a mayúsculas y minúsculas (Edad ≠ edad).
- No pueden tener espacios ni símbolos especiales.

◆ Buenas prácticas

- Usa nombres claros y descriptivos:
- ❌ x, y, z ✅ nombre, edad, precio_total
- Python sigue la convención snake_case: Palabras en minúsculas separadas por guiones bajos → numero_de_alumnos

Conversiones de tipo en Python

¿Qué es una conversión de tipo?

Una conversión de tipo (o casting) consiste en cambiar el tipo de dato de una variable a otro tipo compatible. Python lo permite fácilmente usando funciones predefinidas como **int()**, **float()**, **str()**, **bool()**, etc.

Tipo inicial	Conversión	Resultado
"25" → <code>int("25")</code>	<code>int</code>	25
"3.5" → <code>float("3.5")</code>	<code>float</code>	3.5
5 → <code>str(5)</code>	<code>str</code>	"5"
0 → <code>bool(0)</code>	<code>bool</code>	False



Conversiones de tipo en Python

Python a veces convierte los tipos automáticamente si puede hacerlo sin perder información:

```
x = 5          # int
y = 2.0        # float
resultado = x + y
print(resultado)
```

Salida
7.0

Errores comunes:

No todos los valores se pueden convertir directamente:

```
int("hola")    # ❌ Error: ValueError
```

Solo puedes convertir si el texto representa un número válido.

Notas:

Antes de usar un valor en operaciones numéricas, asegúrate de que sea un número, no una cadena.

Usa `int()` o `float()` cuando obtengas datos con `input()`.



Las variables y referencias en Python

En Python, una variable no guarda el valor directamente, sino que actúa como un identificador que referencia a una posición de memoria donde se almacena ese valor. Es decir, el nombre de la variable funciona como una etiqueta que apunta al lugar en el que vive el dato dentro de la memoria del ordenador.

Cuando escribimos `x = 5`, Python crea un objeto con el valor 5 en una dirección concreta de la memoria y hace que el identificador `x` apunte a ese objeto.

Si después hacemos `y = x`, tanto `x` como `y` apuntan al mismo valor, compartiendo la misma referencia.

En cambio, si reasignamos `x = 10`, el identificador `x` deja de apuntar al 5 y pasa a referenciar un nuevo objeto con el valor 10, mientras que `y` sigue apuntando al 5.

Qué significa “referencia” en este contexto

En Python, todo es un objeto: los números, los textos, las listas, etc.

Cuando creas una variable, lo que se almacena no es el valor, sino una referencia (la dirección de memoria del objeto).

El identificador es el nombre que te permite acceder a ese objeto.



Las variables y referencias en Python

Para entenderlo mejor, escribe y ejecuta el siguiente programa:

```
X = 5
```

```
y = x
```

```
print("la referencia de x está en ", id(x), "y x tiene el valor", x)
```

```
print("La referencia de y está en ", id(y), "e y tiene valor", y)
```

```
Y = 10
```

```
print("Ahora x sigue igual")print("la referencia de x está en ", id(x), "y x tiene el valor", x)
```

```
print("Pero y ha cambiado")print("La referencia de y está en ", id(y), "e y tiene valor", y)
```

En Python, las variables no guardan el valor directamente, sino una referencia al objeto en memoria. Cuando reasignamos una variable, cambia su referencia, pero las demás que apuntaban al mismo valor siguen igual.

Palabras reservadas en Python

Las **palabras reservadas** son las instrucciones y expresiones recogidas en el diccionario de Python.

No se pueden usar como nombres de variables o funciones.

Palabras reservadas de Python

Categoría	Palabras clave
Control de flujo	<code>if</code> , <code>elif</code> , <code>else</code> , <code>for</code> , <code>while</code> , <code>break</code> , <code>continue</code> , <code>pass</code>
Definición y estructura	<code>def</code> , <code>class</code> , <code>return</code> , <code>yield</code> , <code>lambda</code> , <code>with</code> , <code>as</code>
Valores lógicos y nulos	<code>True</code> , <code>False</code> , <code>None</code>
Operadores lógicos y de comparación	<code>and</code> , <code>or</code> , <code>not</code> , <code>in</code> , <code>is</code>
Excepciones y errores	<code>try</code> , <code>except</code> , <code>finally</code> , <code>raise</code> , <code>assert</code>
Importación de módulos	<code>import</code> , <code>from</code> , <code>global</code> , <code>nonlocal</code>
Manipulación de contexto	<code>del</code> , <code>async</code> , <code>await</code>
Otros (especiales o estructurales)	<code>match</code> , <code>case</code> <i>(desde Python 3.10)</i>

Constantes en Python

💡 ¿Qué es una constante?

Una **constante** es un **valor que no debe cambiar** durante la ejecución de un programa. Sirve para **almacenar información fija o que no debería modificarse**.

Por ejemplo:

- El número **pi** (3.1416)
- La cantidad de **días de la semana** (7)
- El **IVA** (0.21)
- O un **mensaje fijo** como `"Bienvenido al programa"`

💎 Python no tiene constantes "reales"

A diferencia de otros lenguajes (como Java o C++),

👉 en **Python no existe una palabra clave** para declarar una constante.

Sin embargo, **por convención**, los programadores usan **nombres en mayúsculas** para indicar que una variable **debería considerarse constante** (aunque técnicamente se pueda cambiar).

Constantes en Python

◆ Ejemplo de constante

```
python

PI = 3.1416
IVA = 0.21
MENSAJE = "Bienvenido al sistema"

print(PI)
print(MENSAJE)
```

🧠 Aquí `PI`, `IVA` y `MENSAJE` se consideran constantes por convención, aunque Python no impide que las cambies:

```
python

PI = 5
print(PI)  # ! Python lo permite, pero está mal visto
```

Constantes en Python

◆ Buenas prácticas

✓ Usar nombres en MAYÚSCULAS y, si hay varias palabras, separarlas con guiones bajos (_):

python

```
MAX_ALUMNOS = 30
TASA_CAMBIO = 0.92
MENSAJE_INICIO = "Hola, usuario"
```

✗ No modificar su valor en el programa.

💡 Esto no es obligatorio para Python, pero sí una **regla de estilo (PEP 8)** aceptada universalmente.

◆ Constantes "simuladas" con módulos

Cuando un proyecto crece, a veces se agrupan las constantes en un archivo aparte:

por ejemplo, `constantes.py`

python

```
# archivo: constantes.py
PI = 3.1416
IVA = 0.21
```

La función `input()` en Python

🧩 La función `input()` en Python

💡 ¿Qué hace?

La función `input()` sirve para pedir datos al usuario por pantalla y guardar lo que escribe en una variable.

Cuando Python encuentra un `input()`, pausa el programa, muestra un mensaje opcional en pantalla, y espera a que el usuario escriba algo y pulse Enter.

💠 Sintaxis básica

```
python

variable = input("Mensaje al usuario: ")
```

🧠 Python muestra el texto "Mensaje al usuario: "

y espera que el usuario escriba algo.

Lo que el usuario escriba se guarda en la variable como **texto** (`str`).

La función input() en Python

◆ Ejemplo simple

python

```
nombre = input("¿Cómo te llamas? ")  
print("Hola", nombre)
```

🖥 Salida:

mathematica

```
¿Cómo te llamas? Ana  
Hola Ana
```

◆ Por defecto, `input()` siempre devuelve texto (`str`)

Aunque el usuario escriba un número, Python lo guarda como texto:

python

```
edad = input("¿Cuántos años tienes? ")  
print(edad + 1)  # ❌ Error
```

La función input() en Python

◆ Cómo convertir el valor a número

Si necesitas hacer cálculos, debes **convertir** el dato al tipo correcto:

Conversión	Tipo de dato	Ejemplo
<code>int()</code>	Entero	<code>edad = int(input("Edad: "))</code>
<code>float()</code>	Decimal	<code>nota = float(input("Nota: "))</code>

💡 Ejemplo completo:

python

```
edad = int(input("¿Cuántos años tienes? "))  
print("El año que viene tendrás", edad + 1)
```

💻 Salida:

css

```
¿Cuántos años tienes? 20  
El año que viene tendrás 21
```

La función input() en Python

♦ Ejemplo con varios `input()`

```
python

nombre = input("Nombre: ")
edad = int(input("Edad: "))
altura = float(input("Altura en metros: "))

print(f"{nombre} tiene {edad} años y mide {altura} m.")
```

Salida:

```
yaml

Nombre: Laura
Edad: 17
Altura en metros: 1.65
Laura tiene 17 años y mide 1.65 m.
```

♦ Usando `input()` sin mensaje

Si no pones texto dentro, Python simplemente espera que el usuario escriba algo:

```
python

dato = input()
```

Esto no es recomendable, porque el usuario no sabrá qué debe introducir.